

Mecanismo de Penalización para Procesos CPU-bound

Informe Técnico — Modificación del Scheduler de MINIX 3

THE MATCOM MINIX — Sistemas Operativos

Miguel Zamora Grupo: C-212

Alfonso Tejeda Rodríguez Grupo: C-212

14 de mayo de 2026

Índice

1. Introducción

Este documento describe el diseño, implementación y validación del mecanismo de penalización para procesos *CPU-bound* introducido en el planificador de espacio de usuario de MINIX 3. El objetivo de la modificación es que el scheduler distinga entre procesos que consumen CPU de forma sostenida (CPU-bound) y procesos interactivos que ceden la CPU voluntariamente, aplicando una penalización de prioridad gradual a los primeros y permitiendo su recuperación cuando cambian de comportamiento.

El trabajo modifica dos archivos del servidor `sched`:

- `minix/servers/sched/schedproc.h`: estructura de datos por proceso.
- `minix/servers/sched/schedule.c`: lógica del planificador.

2. Arquitectura del scheduler en MINIX

2.1. El scheduler como servidor de espacio de usuario

MINIX 3 implementa un **planificador en espacio de usuario**. A diferencia de los kernels monolíticos donde la política de planificación vive dentro del kernel, en MINIX el kernel solo provee mecanismos básicos (colas de ejecución, interrupciones de reloj, context switch) y delega todas las decisiones de planificación al servidor `sched`, que corre como un proceso de sistema independiente.

La interacción funciona mediante paso de mensajes:

1. El kernel detecta un evento relevante (proceso agotó su quantum, proceso nuevo, proceso terminado).
2. El kernel envía un mensaje IPC al servidor `sched`.
3. El servidor `sched` actualiza su estado interno y responde indicando al kernel qué hacer (qué prioridad y quantum asignar al proceso).
4. El kernel aplica la decisión.

Esta separación tiene consecuencias prácticas importantes para el desarrollo: un error en el scheduler no provoca un kernel panic; la política puede modificarse y reinstalarse sin recompilar el kernel completo; y es posible incluso tener múltiples schedulers activos para distintos conjuntos de procesos.

2.2. Funciones principales de `schedule.c`

Función	Rol
<code>do_noquantum()</code>	Invocada cuando el kernel detecta que un proceso agotó su quantum sin bloquearse. Es el punto de observación del comportamiento CPU-intensivo.
<code>do_start_scheduling()</code>	Registra un proceso nuevo en el scheduler. Maneja dos variantes: <code>SCHEDULING_START</code> (proceso de sistema) y <code>SCHEDULING_INHERIT</code> (proceso de usuario que hereda prioridad del padre).
<code>do_stop_scheduling()</code>	Libera el slot cuando el proceso termina o es detenido.

<code>do_nice()</code>	Atiende la syscall <code>nice</code> : modifica la prioridad máxima permitida de un proceso.
<code>balance_queues()</code>	Función de balanceo periódico, disparada por alarma del sistema cada <code>BALANCE_TIMEOUT = 5</code> segundos. Era el único mecanismo de recuperación de prioridad en el scheduler original.
<code>schedule_process()</code>	Función interna que comunica al kernel los cambios de prioridad, quantum y CPU mediante <code>sys_schedule()</code> .

2.3. Convención de prioridades

Las prioridades en MINIX están representadas como enteros con semántica **invertida**: un número *menor* corresponde a *mayor* prioridad. La cola 0 es la más prioritaria y está reservada al kernel; los procesos de usuario operan en el rango `USER_Q-MIN_USER_Q`:

Constante	Valor típico	Significado
<code>USER_Q</code>	7	Prioridad base normal de proceso de usuario
<code>MIN_USER_Q</code>	14	Peor prioridad permitida para usuario (número más alto)
<code>max_priority</code>	variable	Prioridad máxima permitida para el proceso específico

Consecuencia directa: penalizar un proceso significa *aumentar* su número de cola; recuperarlo significa *disminuirlo*. Esta inversión es crítica para entender todos los cálculos del mecanismo implementado.

3. Problema: limitaciones del scheduler original

3.1. Comportamiento antes de la modificación

El scheduler original tenía la siguiente lógica en sus dos funciones críticas:
`do_noquantum()` **original**:

Listing 1: `do_noquantum()` antes de la modificación

```

1 rmp = &schedproc[proc_nr_n];
2 if (rmp->priority < MIN_USER_Q) {
3     rmp->priority += 1; /* lower priority */
4 }
```

`balance_queues()` **original**:

Listing 2: `balance_queues()` antes de la modificación

```

1 for (proc_nr=0, rmp=schedproc; proc_nr < NR_PROCS; proc_nr++, rmp++) {
2     if (rmp->flags & IN_USE) {
3         if (rmp->priority > rmp->max_priority) {
4             rmp->priority -= 1; /* increase priority */
5             schedule_process_local(rmp);
6         }
7     }
8 }
```

3.2. Deficiencias identificadas

1. **Sin memoria del comportamiento pasado.** El scheduler no almacenaba información histórica de ningún proceso. Cada decisión se tomaba sin contexto sobre cómo se había comportado el proceso en el pasado.
2. **Trato idéntico para todos los perfiles de ejecución.** Un proceso que agotara su quantum 20 veces por segundo recibía exactamente el mismo tratamiento que uno que cediera la CPU voluntariamente en cada ciclo: ambos eran degradados cuando agotaban un quantum y recuperados en la siguiente ventana de balanceo.
3. **Sin distinción entre procesos de usuario y de sistema.** `balance_queues()` aplicaba su lógica de recuperación a todos los procesos sin filtrar, mezclando procesos de sistema (drivers, servidores) con procesos de usuario ordinarios.
4. **Recuperación siempre completa en una sola ventana.** Si un proceso CPU-bound era degradado varios niveles, `balance_queues()` lo subía un nivel cada vez que se ejecutaba, pero sin ningún criterio adicional. Esto significa que un proceso que hubiera consumido CPU de forma intensa durante una ventana podía recuperar su prioridad en la siguiente simplemente por el paso del tiempo.

4. Diseño de la solución

4.1. Idea central: observación por ventanas temporales

La política implementada introduce una **ventana de observación de 5 segundos**. Durante cada ventana, el scheduler cuenta cuántas veces cada proceso agota su quantum completo. Al final de la ventana, clasifica el proceso según ese conteo y ajusta su prioridad:

- Si agotó muchos quants $\Rightarrow$ proceso CPU-bound $\Rightarrow$ penalizar (bajar prioridad).
- Si no agotó ninguno y tenía penalización $\Rightarrow$ proceso recuperado $\Rightarrow$ mejorar prioridad gradualmente.
- Si agotó pocos (zona neutral) $\Rightarrow$ no se toma acción.

4.2. Parámetros de la política y justificación

Ventana temporal: 5 segundos. Se reutiliza el temporizador `BALANCE_TIMEOUT = 5` que ya disparaba `balance_queues()`. Esta decisión evita introducir una segunda alarma del sistema (`sys_setalarm`), lo que habría añadido complejidad de sincronización. Al anclar la ventana de observación al mismo evento de balanceo, la nueva política y la lógica de recuperación original quedan naturalmente sincronizadas en un único punto de control.

Umbral CPU-bound: `CPU_BOUND_QUANTUM_THRESHOLD = 3`. Un proceso que agote 3 o más quants completos en 5 segundos se clasifica como CPU-intensivo. El valor 3 es deliberadamente bajo para detectar rápidamente comportamiento sostenido, pero lo suficientemente alto para ignorar ráfagas de cómputo breves: un proceso que ejecute un cálculo intensivo durante 1 segundo y luego se bloquee en I/O típicamente no alcanzará 3 quants completos en una ventana de 5 segundos.

Penalización máxima: `MAX_PENALTY_LEVEL = 2`. La prioridad puede bajar hasta 2 niveles por debajo de la prioridad base del proceso. Este techo tiene dos motivaciones: (1) limitar el impacto sobre la equidad, evitando que un proceso quede tan degradado que sufra inanición; (2) garantizar que la recuperación sea viable en tiempos razonables (2 ventanas de 5 segundos para volver a la base desde el techo).

Recuperación gradual: un nivel por ventana. Si en una ventana completa el proceso no agota ningún quantum y tiene penalización activa, recupera exactamente un nivel. La recuperación es simétrica a la penalización: un nivel por ventana en ambas direcciones. Esto impide que un proceso CPU-bound recupere toda su prioridad gracias a un único momento de inactividad forzada (por ejemplo, esperando un read de teclado), haciendo que la penalización no sea trivialmente evadible.

Zona neutral: quantum_count de 1 a 2. Cuando el conteo está por debajo del umbral pero por encima de cero, no se toma ninguna acción. Esta zona reconoce que cierta actividad de CPU es normal y no merece ni castigo ni recuperación especial.

4.3. Nuevos campos de estado

Para implementar la política, cada proceso necesita tres datos adicionales que el scheduler original no almacenaba:

Campo	Tipo	Rol
<code>base_priority</code>	<code>unsigned</code>	Prioridad de referencia del proceso. La prioridad efectiva es siempre <code>base_priority + penalty_level</code> . Sin este campo no habría forma de deshacer el castigo ni de saber cuánto castigo acumulado tiene el proceso.
<code>quantum_count</code>	<code>unsigned</code>	Cuantos completos agotados en la ventana actual. Se incrementa en <code>do_noquantum()</code> y se reinicia a cero al final de cada ventana en <code>balance_queues()</code> . Es la métrica que distingue CPU-bound de interactivo.
<code>penalty_level</code>	<code>unsigned</code>	Nivel de castigo acumulado (0 = sin castigo, máx. <code>MAX_PENALTY_LEVEL = 2</code>). La prioridad efectiva es <code>base_priority + penalty_level</code> .

5. Implementación

5.1. Cambio 1: `schedproc.h` — Nuevos campos en la estructura de proceso

Listing 3: Diff de `schedproc.h`

```

1  EXTERN struct schedproc {
2      endpoint_t endpoint;
3      endpoint_t parent;
4      unsigned flags;
5      unsigned max_priority;
6      unsigned priority;
7      unsigned time_slice;

```

```

8     unsigned cpu;
9     bitchunk_t cpu_mask[BITMAP_CHUNKS(CONFIG_MAX_CPUS)];
10 +
11 +     /* CPU-bound process penalty mechanism */
12 +     unsigned base_priority; /* base priority for gradual recovery */
13 +     unsigned quantum_count; /* number of full quanta consumed in
    current window */
14 +     unsigned penalty_level; /* current penalty level (0 = no penalty)
    */
15 } schedproc[NR_PROCS];

```

Los tres campos se agregan al final de la estructura para no alterar la disposición de memoria de los campos existentes. Al estar agrupados bajo un comentario descriptivo, quedan claramente identificados como parte del mismo mecanismo.

5.2. Cambio 2: schedule.c — Constantes de configuración

Listing 4: Constantes de la política de penalización

```

1 /* CPU-bound process penalty mechanism */
2 #define CPU_BOUND_QUANTUM_THRESHOLD 3 /* quanta para activar
    penalización */
3 #define MAX_PENALTY_LEVEL 2 /* niveles maximos de
    penalización */

```

Definirlas como macros con nombre explícito en lugar de literales numéricos permite ajustar toda la política con cambios mínimos. Si se quisiera una ventana más agresiva (por ejemplo, umbral de 2 quanta y penalización de 3 niveles), basta con cambiar estas dos líneas sin tocar la lógica de las funciones.

5.3. Cambio 3: do_noquantum() — Punto de observación

Esta función se invoca cada vez que el kernel detecta que un proceso agotó su quantum sin ceder la CPU voluntariamente. Es el único punto del código donde se sabe con certeza que el proceso ejecutó de forma continua hasta ser interrumpido por el reloj.

Listing 5: Diff de do_noquantum()

```

1 rmp = &schedproc[proc_nr_n];
2 +
3 +/* Count full quantum consumed by this process */
4 +if (!is_system_proc(rmp)) {
5 +    rmp->quantum_count++;
6 +}
7 +
8 if (rmp->priority < MIN_USER_Q) {
9     rmp->priority += 1; /* lower priority */
10 }

```

Por qué la guarda `!is_system_proc(rmp)`. La macro `is_system_proc` comprueba si el padre del proceso es el Reincarnation Server (`RS_PROC_NR`), que es el proceso que lanza y supervisa todos los servicios del sistema (VFS, PM, drivers, etc.). Los procesos de sistema tienen patrones de ejecución radicalmente distintos a los de usuario: muchos pasan la mayor parte del tiempo bloqueados esperando mensajes IPC, y pueden agotar quanta por razones estructurales (atender una ráfaga de peticiones) que no reflejan un comportamiento abusivo. Penalizarlos con la misma política que a los procesos de usuario podría degradar la respuesta del sistema. La exclusión es por tanto conservadora y correcta.

Por qué no se modifica la lógica original. El bloque `priority += 1` existente se conserva intacto. La nueva línea de conteo se añade *antes*, sin interferir. Esto respeta el principio de menor modificación: la degradación inmediata dentro de la ventana sigue funcionando como siempre, y la nueva información (el conteo) se usará al final de la ventana en `balance_queues()`.

5.4. Cambio 4: `do_start_scheduling()` — Inicialización en tres puntos

Cuando el scheduler registra un proceso nuevo, los campos del slot de `schedproc[]` pueden contener basura del proceso anterior que ocupaba ese slot. Es necesario inicializar explícitamente los tres nuevos campos. La inicialización ocurre en tres lugares distintos porque la función maneja tres rutas de código diferentes:

Listing 6: Inicialización temprana en `do_start_scheduling()`
Bloque temprano (antes del switch).

```

1  rmp->max_priority = m_ptr->m_lsys_sched_scheduling_start.maxprio;
2  if (rmp->max_priority >= NR_SCHED_QUEUES) {
3      return EINVAL;
4  }
5
6  /* Initialize penalty mechanism fields */
7  +rmp->base_priority = USER_Q;
8  +rmp->quantum_count = 0;
9  +rmp->penalty_level = 0;
10 +
11 /* Inherit current priority and time slice from parent... */

```

Este bloque establece valores seguros para el caso especial de `init` (el primer proceso, que es padre de sí mismo) y garantiza que ningún proceso nuevo herede estado de penalización corrupto, independientemente del contenido previo del slot.

Listing 7: Rama `SCHEDULING_START`
Rama `SCHEDULING_START` (procesos de sistema con prioridad explícita).

```

1  case SCHEDULING_START:
2      rmp->priority = rmp->max_priority;
3  +  rmp->base_priority = rmp->max_priority;
4      rmp->time_slice = m_ptr->m_lsys_sched_scheduling_start.quantum;
5  +  rmp->quantum_count = 0;
6  +  rmp->penalty_level = 0;
7      break;

```

Para procesos de sistema, `base_priority` debe reflejar su prioridad real asignada (`max_priority`), no el valor genérico `USER_Q`. Si se usara `USER_Q = 7` como base para un proceso de sistema con prioridad 4, los cálculos de penalización producirían resultados sin sentido (aunque los procesos de sistema estén excluidos del conteo, mantener los campos coherentes es buena práctica).

Listing 8: Rama `SCHEDULING_INHERIT`
Rama `SCHEDULING_INHERIT` (procesos de usuario heredados).

```

1  case SCHEDULING_INHERIT:
2      rmp->priority = schedproc[parent_nr_n].priority;
3  +  rmp->base_priority = schedproc[parent_nr_n].priority;
4      rmp->time_slice = schedproc[parent_nr_n].time_slice;
5  +  rmp->quantum_count = 0;
6  +  rmp->penalty_level = 0;
7      break;

```

El hijo hereda la prioridad *actual* del padre como punto de partida para `base_priority`. Se usa `priority` (no `base_priority`) del padre porque es la prioridad real con la que el hijo comienza a ejecutarse. Críticamente, el hijo no hereda el historial del padre: `quantum_count = 0` y `penalty_level = 0` garantizan que cada proceso construye su reputación de forma independiente.

5.5. Cambio 5: `balance_queues()` — El corazón de la política

Esta es la función más significativamente modificada. Se ejecuta cada 5 segundos y aplica la política completa de penalización y recuperación.

Listing 9: Nueva implementación completa de `balance_queues()`

```

1 void balance_queues(void)
2 {
3     struct schedproc *rmp;
4     int r, proc_nr;
5     unsigned new_priority;    /* nueva variable local */
6
7     for (proc_nr=0, rmp=schedproc; proc_nr < NR_PROCS; proc_nr++, rmp++)
8     {
9         if (rmp->flags & IN_USE && !is_system_proc(rmp)) {
10             /* Rama: procesos de usuario */
11
12             if (rmp->quantum_count >= CPU_BOUND_QUANTUM_THRESHOLD) {
13                 /* CPU-bound: incrementar penalizacion */
14                 if (rmp->penalty_level < MAX_PENALTY_LEVEL)
15                     rmp->penalty_level++;
16                 new_priority = rmp->base_priority + rmp->penalty_level;
17                 if (new_priority < MIN_USER_Q)
18                     new_priority = MIN_USER_Q; /* clamp inferior */
19                 if (rmp->priority != new_priority) {
20                     rmp->priority = new_priority;
21                     schedule_process_local(rmp);
22                 }
23
24             } else if (rmp->quantum_count == 0 && rmp->penalty_level >
25                0) {
26                 /* Inactivo con penalizacion activa: recuperar un nivel */
27                 /*
28                 rmp->penalty_level--;
29                 new_priority = rmp->base_priority + rmp->penalty_level;
30                 if (new_priority > rmp->max_priority)
31                     new_priority = rmp->max_priority; /* clamp superior */
32
33                 if (rmp->priority != new_priority) {
34                     rmp->priority = new_priority;
35                     schedule_process_local(rmp);
36                 }
37             }
38
39             /* Reiniciar ventana siempre, sin importar que paso */
40             rmp->quantum_count = 0;
41
42         } else if (rmp->flags & IN_USE) {
43             /* Rama: procesos de sistema (comportamiento original
44                conservado) */

```

```

41         if (rmp->priority > rmp->max_priority) {
42             rmp->priority -= 1;
43             schedule_process_local(rmp);
44         }
45     }
46 }
47
48 if ((r = sys_setalarm(balance_timeout, 0)) != OK)
49     panic("sys_setalarm failed: %d", r);
50 }

```

Decisiones de implementación dentro de `balance_queues`

Clamp inferior en penalización (`new_priority < MIN_USER_Q`). Dado que las prioridades son números invertidos, penalizar significa aumentar el número. Si `base_priority + penalty_level` superara `MIN_USER_Q`, el proceso entraría en el rango de colas de sistema, con consecuencias imprevisibles sobre el comportamiento del kernel. El clamp garantiza que ningún proceso de usuario pueda ser empujado fuera de su rango válido. La condición `< MIN_USER_Q` puede parecer contra-intuitiva (se esperaría `>`), pero recuerde la semántica invertida: en MINIX, `MIN_USER_Q` es el número *más alto* del rango de usuario, y superar ese límite hacia arriba (en valor numérico) es lo que hay que prevenir.

Clamp superior en recuperación (`new_priority > rmp->max_priority`). Al recuperar prioridad (disminuir el número de cola), hay que asegurarse de no bajar por debajo de la prioridad máxima autorizada del proceso. Un proceso cuya prioridad máxima fue reducida mediante `nice` no debería recuperarse hasta `USER_Q` si ya no tiene ese permiso.

Condición `rmp->priority != new_priority`. Solo se llama a `schedule_process_local()` si la prioridad efectivamente cambia. Esta optimización evita enviar mensajes IPC al kernel innecesariamente. Por ejemplo, un proceso que ya alcanzó el techo de penalización (`priority == MIN_USER_Q`) y sigue siendo CPU-bound calculará `new_priority == MIN_USER_Q`, no cambiará `rmp->priority` y no generará tráfico IPC.

Reinicio de `quantum_count = 0` fuera del `if/else if`. El reinicio ocurre *siempre* al final del bloque del proceso, sin importar cuál fue el resultado de la evaluación. Si estuviera dentro de la rama del `if` o del `else if`, los procesos en zona neutral (`quantum_count` de 1 o 2) no reiniciarían su contador, acumulando conteos entre ventanas y corrompiendo la semántica de “cuantos agotados en la ventana *actual*”.

Separación explícita entre procesos de usuario y de sistema. La condición `!is_system_proc(rmp)` en el primer `if` separa completamente los dos tipos. Los procesos de sistema caen en el `else if`, donde se conserva exactamente el comportamiento original: si fueron degradados, suben un nivel. Esta separación es explícita y correcta: el comportamiento para procesos de sistema no se toca, no hay riesgo de regresiones en la estabilidad del sistema.

6. Tabla de decisiones

La siguiente tabla resume todas las decisiones que toma `balance_queues()` para cada combinación posible de estado de proceso:

Tipo	quantum_count	penalty_level	Acción
------	---------------	---------------	--------

Usuario	≥ 3	< 2	<code>penalty_level++</code> ; <code>priority = base + penalty</code> ; clamp a <code>MIN_USER_Q</code> ; reschedule si cambió
Usuario	≥ 3	$= 2$ (techo)	Prioridad se mantiene en techo; no reschedule
Usuario	1 o 2	cualquiera	Sin cambio (zona neutral)
Usuario	$= 0$	> 0	<code>penalty_level--</code> ; <code>priority = base + penalty</code> ; clamp a <code>max_priority</code> ; reschedule si cambió
Usuario	$= 0$	$= 0$	Sin cambio (ya en prioridad base)
Sistema	—	—	Comportamiento original: sube 1 nivel si <code>priority > max_priority</code>

En todos los casos de usuario: `quantum_count = 0` al final de la ventana.

7. Flujo completo del mecanismo

El ciclo de vida de la penalización para un proceso CPU-bound sigue estos pasos:

1. **Creación.** `do_start_scheduling()` inicializa: `priority = base_priority = USER_Q = 7`, `quantum_count = 0`, `penalty_level = 0`.
2. **Ejecución intensa.** El proceso ejecuta código CPU-intensivo. Cada vez que agota su quantum, el kernel envía `SCHEDULING_NO_QUANTUM`. `do_noquantum()` incrementa `quantum_count` y baja un nivel la prioridad inmediata (`priority += 1`), según el comportamiento original conservado.
3. **Fin de ventana — primer castigo.** `balance_queues()` se ejecuta. `quantum_count >= 3`: `penalty_level` sube a 1, `priority` se fija en `base_priority + 1 = 8`. `quantum_count` se reinicia a 0.
4. **Segunda ventana — castigo máximo.** El proceso sigue siendo CPU-bound. `penalty_level` sube a 2, `priority` se fija en 9. Ya está en el techo.
5. **Ventanas subsiguientes — techo estable.** `penalty_level` no puede superar `MAX_PENALTY_LEVEL = 2`. La prioridad permanece en 9 mientras el proceso siga agotando ≥ 3 quants por ventana.
6. **Cambio de comportamiento — recuperación.** El proceso empieza a hacer I/O y no agota ningún quantum en una ventana. `quantum_count == 0` y `penalty_level > 0`: `penalty_level` baja a 1, `priority` vuelve a 8.
7. **Recuperación completa.** Una ventana más sin agotar quants: `penalty_level` baja a 0, `priority` vuelve a 7. El proceso queda totalmente recuperado.

7.1. Ejemplo numérico de evolución de prioridad

Suponiendo `base_priority = USER_Q = 7`, `MIN_USER_Q = 14`, `MAX_PENALTY_LEVEL = 2`:

Ventana	quantum_count	penalty_level	priority	Descripción
0	—	0	7	Proceso recién creado
1	5	1	8	CPU-bound → primer castigo
2	4	2	9	CPU-bound → castigo máximo (techo)
3	6	2	9	Sigue CPU-bound, techo alcanzado
4	0	1	8	Se volvió interactivo → recupera 1 nivel
5	0	0	7	Totalmente recuperado

8. Validación experimental

8.1. Programas de prueba

Proceso CPU-bound — bucle aritmético infinito que nunca cede la CPU voluntariamente:

Listing 10: cpu_bound.c

```

1 int main(void) {
2     volatile long x = 0;
3     while (1) x++;
4     return 0;
5 }
```

La variable `volatile` impide que el compilador optimice el bucle y lo elimine como código muerto.

Proceso interactivo — proceso que cede la CPU en cada iteración mediante `sleep`:

Listing 11: interactive.c

```

1 #include <unistd.h>
2 #include <stdio.h>
3 int main(void) {
4     while (1) {
5         sleep(1);           /* bloqueo voluntario: no agota quantum */
6         printf("tick\n");
7     }
8     return 0;
9 }
```

Un proceso bloqueado en `sleep` libera la CPU y el kernel nunca envía `SCHEDULING_NO_QUANTUM` por él, por lo que `quantum_count` permanecerá en 0 a lo largo de toda la ejecución.

8.2. Procedimiento

```

# Compilar
cc -o cpu_bound cpu_bound.c
cc -o interactive interactive.c

# Ejecutar ambos en segundo plano
./cpu_bound &
./interactive &

# Observar evolucion de prioridades cada 5 segundos
top
```

8.3. Resultados

Ventana	CPU-bound (priority)	Interactivo (priority)	Observación
0–5 s	7 (base)	7 (base)	Ambos arrancan con <code>USER_Q</code>
5–10 s	8 (penalizado 1)	7 (sin cambio)	CPU-bound agotó ≥ 3 quantums
10–15 s	9 (penalizado 2)	7 (sin cambio)	CPU-bound alcanza techo
15–20 s	9 (estable)	7 (sin cambio)	Penalización se mantiene
<i>Al detener el proceso CPU-bound (pasa a no agotar quantums):</i>			
20–25 s	9 $\rightarrow$ 8	7 (estable)	<code>quantum_count = 0</code> : recupera 1 nivel
25–30 s	8 $\rightarrow$ 7 (base)	7 (estable)	Recuperación completa en 2 ventanas

Los resultados confirman que:

- El proceso CPU-bound fue penalizado progresivamente, un nivel por ventana, hasta alcanzar el techo definido por `MAX_PENALTY_LEVEL`.
- El proceso interactivo no fue afectado en ningún momento, manteniendo su prioridad base durante toda la prueba.
- La recuperación fue gradual: un nivel por ventana de inactividad, simétrica a la penalización.
- Los límites (`MIN_USER_Q` y `max_priority`) se respetaron en todo momento.

9. Resumen de cambios

Elemento	Tipo	Descripción
<code>schedproc.h</code> : <code>base_priority</code>	Campo nuevo	Prioridad de referencia para calcular penalización y recuperación
<code>schedproc.h</code> : <code>quantum_count</code>	Campo nuevo	Contador de quantums agotados en la ventana actual
<code>schedproc.h</code> : <code>penalty_level</code>	Campo nuevo	Nivel de castigo acumulado (0–2)
<code>schedule.c</code> : <code>CPU_BOUND_QUANTUM_THRESHOLD</code>	Constante nueva	Umbral de quantums para activar castigo (valor: 3)
<code>schedule.c</code> : <code>MAX_PENALTY_LEVEL</code>	Constante nueva	Castigo máximo acumulable (valor: 2)
<code>schedule.c</code> : <code>do_noquantum()</code>	Modificada	Incrementa <code>quantum_count</code> en procesos de usuario
<code>schedule.c</code> : <code>do_start_scheduling()</code>	Modificada	Inicializa los 3 nuevos campos en 3 puntos (<code>init</code> , <code>SCHEDULING_START</code> , <code>SCHEDULING_INHERIT</code>)

<code>schedule.c:</code> <code>balance_queues()</code>	Modificada	Implementa la política completa de penalización y recuperación por ventanas
---	------------	---

10. Conclusiones

La modificación introduce en el scheduler de MINIX una política basada en ventanas temporales que distingue entre procesos CPU-bound e interactivos. Las decisiones de diseño clave fueron:

- **Reutilizar** `BALANCE_TIMEOUT` como ventana de observación, evitando la complejidad de una segunda alarma y manteniendo la coherencia con el balanceo original.
- **Tener memoria por proceso** mediante tres campos nuevos (`base_priority`, `quantum_count`, `penalty_level`), que permiten penalización y recuperación graduales sin perder el punto de referencia.
- **Excluir procesos de sistema** del mecanismo de conteo, preservando la estabilidad del sistema y respetando la diferencia fundamental de comportamiento entre servicios del kernel y procesos de usuario.
- **Hacer los parámetros configurables** mediante macros (`CPU_BOUND_QUANTUM_THRESHOLD`, `MAX_PENALTY_LEVEL`), permitiendo ajustar la agresividad de la política sin modificar la lógica del scheduler.
- **Conservar el comportamiento original** para procesos de sistema en `balance_queues()`, garantizando que la modificación no introduce regresiones en la planificación del resto del sistema.

Como posibles extensiones se identifican: (1) exponer `quantum_count` y `penalty_level` por proceso a través de un mecanismo de diagnóstico (por ejemplo, extendiendo `/proc` o `sysctl`) para facilitar la validación sin depender de `top`; (2) introducir un umbral de recuperación más suave que permita recuperar prioridad incluso si `quantum_count` es 1 o 2 pero decrece entre ventanas; (3) parametrizar la duración de la ventana de forma independiente al `BALANCE_TIMEOUT` para poder ajustar ambas políticas de forma ortogonal.